Assignment 1

Fabian Blasch, Nik Gabrovsek, Patrick Gyasi, Gregor Steiner

27.05.2021

Time Series Analysis

1

We can use the **quantmod** package to download the data from the FRED database.

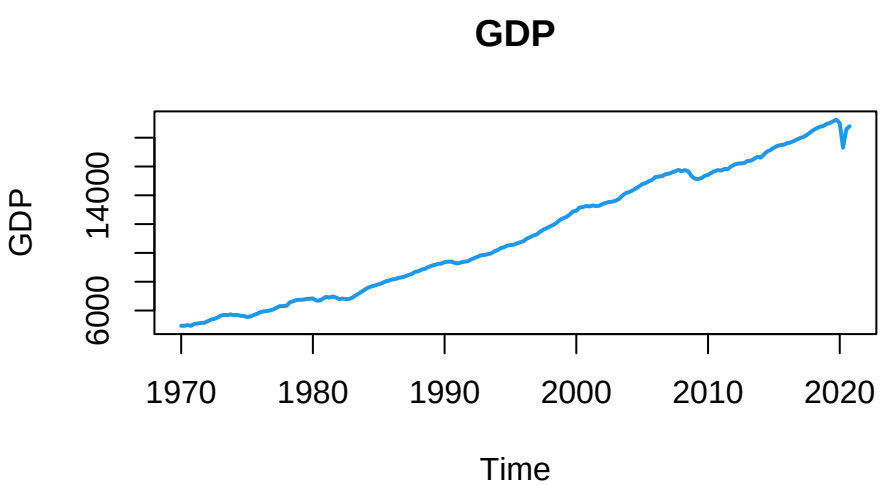
```
# Download data
quantmod::getSymbols("GDPC1", src = "FRED")
```

```
## [1] "GDPC1"
```

```
gdp.ts <- ts(GDPC1, start = c(1947, 1), frequency = 4)

# Filter
gdp.ts <- window(gdp.ts, start = c(1970, 1), end = c(2020, 4))
```

After filtering for the time period we are interested in, we can now plot the series.

```
# Plot
plot(gdp.ts, lwd = 2, col = 4, ylab = "GDP", main = "GDP")
```



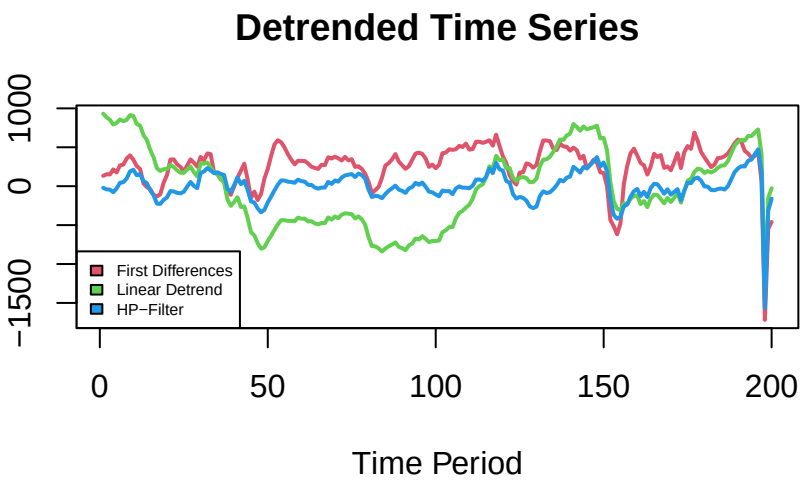
A trend is a slow change that occurs over a long period of time. In our case the trend is clearly positive. The Cyclical component is movement around our trend line, it's of short term nature. Time series with trend in it are not stationary. Stationary time series have no predictable patterns in the long term which is clearly not the case for GDP.

2

The Main idea of detrending is to get rid of a trend in a time series. Detrending changes our un-stationary time series into a stationary one. We know different ways of detrending: first-differences, computing growth rates, linear detrending and filters. The first two are self-explanatory, linear detrending works as follows: if we assume our trend is linear, we can estimate linear regression of our time-series, this gives us a trend and we can then estimate our detrended time-series via the deviations from the linear regression line. Lastly, we can use filters to detrend data, there are various different filter such as the HP and the Bandpass filter. They are similar to linear detrending, but we achieve a smoother fit as the trend line is not necessarily linear.

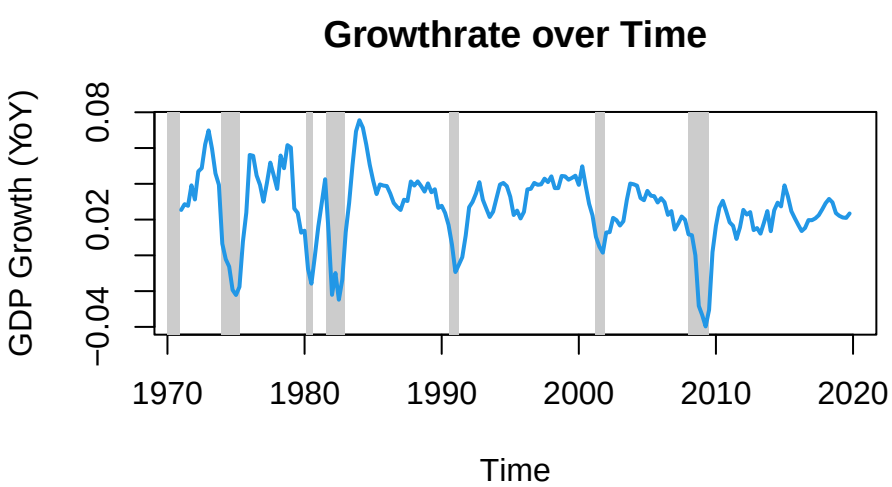
We can plot the three differently detrended lines for GDP and compare them. We use first-differences instead of growth rates here, since the latter will be used later on anyways.

```r
# first differences
gdp.diff <- diff(gdp.ts, 4)

# linear trend
data <- data.frame(gdp = gdp.ts, t = 1:length(gdp.ts))
trend <- fitted(lm(GDPC1 ~ t, data = data))
gdp.linear.detrend <- gdp.ts - trend

# hpfilter
gdp.hp.cycle <- mFilter::hpfilter(gdp.ts, type = "lambda", freq = 1600)$cycle

# Plot
m <- cbind(gdp.diff,
           gdp.linear.detrend[-c(1:4)],
           gdp.hp.cycle[-c(1:4)])

# colours
cols <- 2:4

# plot
matplot(m, type = "l", col = cols, lty = 1, lwd = 2, ylab = "", xlab = "Time Period",
        main = "Detrended Time Series")
legend("bottomleft", fill = cols,
       legend = c("First Differences", "Linear Detrend", "HP-Filter"), cex = 0.5)
```



3

```
# Compute y/y growth rates
gdp.growth <- diff(gdp.ts, 4) / lag(gdp.ts, 4)


# Plot
plot(gdp.growth, type = "n", ylab = "GDP Growth (YoY)", main = "Growthrate over Time")
tis::nberShade()
lines(gdp.growth, lwd = 2, col = 4)
```



If one defines a business cycle as the window of time from one peak of an expansion to the next peak of an expansion, and in the meanwhile the economy undergoes a recession, then the NBER recession dates coincide with the business cycles that can be observed in the plotted time series.

4

```r
# Test for stationarity
tseries::adf.test(gdp.growth)
```

```
## Warning in tseries::adf.test(gdp.growth): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  gdp.growth
## Dickey-Fuller = -4.5746, Lag order = 5, p-value = 0.01
## alternative hypothesis: stationary
```

We use the well-known Augmented Dickey-Fuller Test to test for stationarity. The null-hypothesis of non-stationarity is rejected. Thus, the series seems to be stationary.

5

We want to estimate the model

$$y_t = c + \varphi_1 y_{t-1} + \varphi_2 y_{t-2} + \varepsilon_t$$

in a Bayesian fashion. In matrix notation this reads as

$$y = X\varphi + \varepsilon,$$

where $y := [y_3, ..., y_T]', \varphi := [c, \varphi_1, \varphi_2]', \varepsilon := [\varepsilon_3, ..., \varepsilon_T]'$ and

$$X := \begin{bmatrix} 1 & y_2 & y_1 \\ \vdots & \vdots & \vdots \\ 1 & y_{T-1} & y_{T-2} \end{bmatrix}.$$

We obviously lose the first two observations, leaving us with $T-2$ observations.

The first step is choosing prior distributions for c, φ_1, φ_2. A simple normal prior might be a good start. Thus,

$$\varphi \sim N_3(\mu_0, \Sigma_0),$$

where $\mu_0 = [\mu_c, \mu_{\varphi_1}, \mu_{\varphi_2}]'$ and $\Sigma_0 = diag(\sigma_c^2, \sigma_{\varphi_1}^2, \sigma_{\varphi_2}^2)$.

Then, we only need a prior distribution for the error variance, where we will use an Inverse Gamma Distribution as in the linear model:

$$\sigma^2 \sim IG(c_0, d_0).$$

And we know that

$$y|\varphi, \sigma^2 \sim N_{T-2}(X\varphi, \sigma^2 I_{T-2}).$$

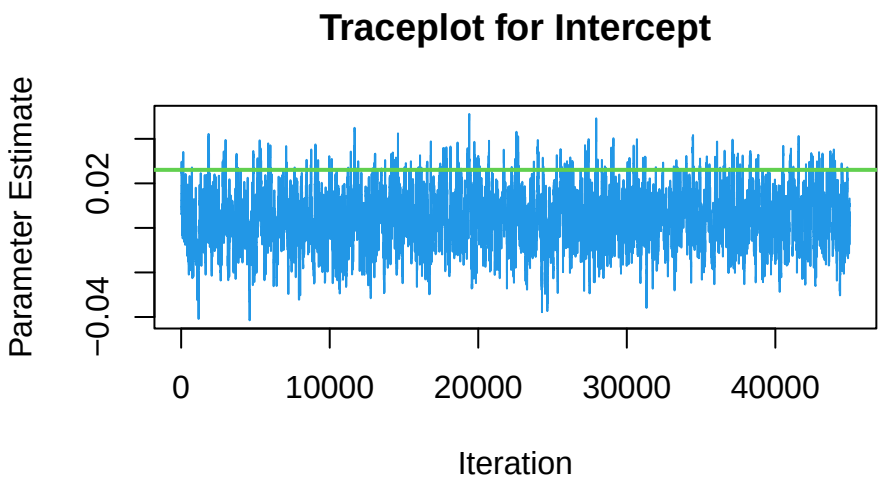
Now, we can start setting up the Metropolis algorithm to estimate the posterior density.

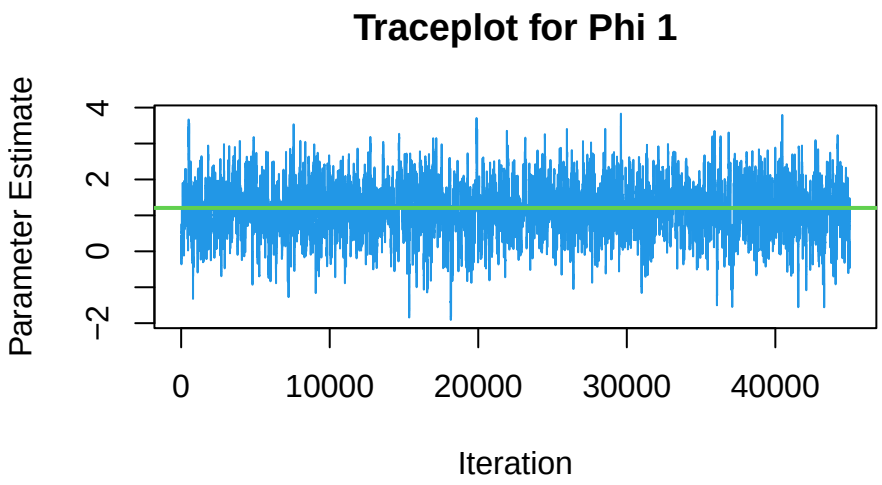
```r
# Set up dependant variable
# time period
t <- length(gdp.growth)


# lags
interc <- rep(1, t - 2)
```

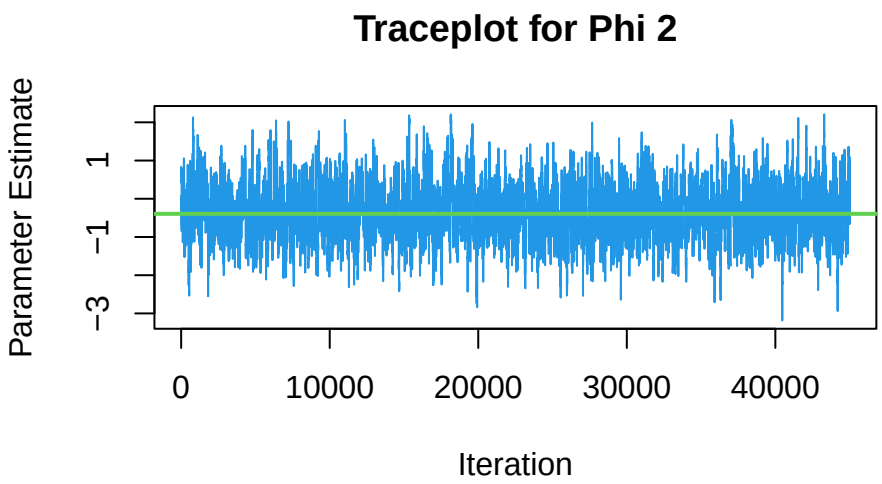
```r
# we remove the first two obs since the L(2) var start in t = 2
y1 <- gdp.growth[-(1:2)]

# lagged vars
y_l1 <- gdp.growth[-c(1, t)]
y_l2 <- gdp.growth[-c(t - 1, t)]

# bind to a matrix for regression
X <- cbind(interc, y_l1, y_l2)

# Metropolis - Hastings algorithm #

# Normal Prior on c
mu0 <- 0
sig0 <- sqrt(100)

# IG prior on on the variance
s0 <- 2
r0 <- 1

# normal prior on phi
pmu0 <- 0
sig0 <- sqrt(100)

# Proposal variances
eta2.c <- 0.005
eta2.phi <- 0.4
eta2 <- c(eta2.c, eta2.phi, eta2.phi)
# storage and starting values
sig2.draw <- 1
phi.draw <- c(0, 0, 0)
accept <- 0
sig0 <- sqrt(100)

# Function wrapper
MCM <- function(nsave = 45000, nburn = 10000, covmat = X, y = y1){
  # storage
  phi.store  <- matrix(NA, nsave, 3)
  sig2.store <- matrix(NA, nsave, 1)
  ntot  <- nsave + nburn

  # loop
  for(i in 1:ntot){
    # create a proposal for phi
    phi.prop <- mapply(function(x, y) rnorm(1, x, y), phi.draw, eta2)

    # evaluate the uniform prior for current draw and proposal
    prior.current <- sum(dnorm(phi.draw[1], mu0, sig0, log = T),
                         dnorm(phi.draw[2:3], pmu0, sig0, log = T))

    prior.prop    <- sum(dnorm(phi.prop[1], mu0, sig0, log = T),
                         dnorm(phi.prop[2:3], pmu0, sig0, log = T))
```

```r
  # evaluate the normal likelihood for current draw and proposal
  ll.current    <- sum(dnorm(y, X %*% phi.draw, sqrt(sig2.draw), log = T))
  ll.prop       <- sum(dnorm(y, X %*% phi.prop, sqrt(sig2.draw), log = T))

  # log likelihood ratio and difference
  c.p.current   <- ll.current + prior.current
  c.p.prop      <- ll.prop    + prior.prop

  # finally, we can compute the likelihood ratio
  ll.ratio      <- c.p.prop - c.p.current

  # Discard if ll smaller than draw
  if(ll.ratio > log(runif(1))){ # if our proposal is likely "enough"

    #replace the current draw for phi with our proposal
    phi.draw <- phi.prop

    #add 1 to our acceptance counter
    accept   <- accept + 1
  }

  # Tune proposal variance during first 50% of burn-in
  if (i < 0.5 * nburn) {
    if(accept / i > 0.3) eta2 <- eta2 * 1.01
    if(accept / i < 0.15) eta2 <- eta2 * 0.99
  }

   # STEP 2: SAMPLE SIGMA2 GIVEN phi & DATA

  # compute errors
  e  <- y - X %*% phi.draw
  # sum of squared residuals (equivalent to sum((e)^2))
  ee <- t(e) %*% e

  # compute posterior quantities
  s <- s0 +  t/2
  r <- r0 + ee/2

  # sample sigma2 from inverse gamma posterior
  sig2.draw <- 1 / rgamma(1, s, r)

# STEP 3: STORE POSTERIOR SAMPLES

  #only after burn-in period!

  if(i > nburn){

  phi.store[i - nburn,]  <- phi.draw
  sig2.store[i - nburn,] <- sig2.draw

  }
}
```

```r
  return(list(phi.store,
              accept / i))
}

# run
mcm1 <- MCM()

# Estimate by ML to compare
phi.mle <- arima(y1, order = c(2L, 0L, 0L), include.mean = TRUE)$coef[c(3, 1:2)]

# traceplot
invisible(
Map(function(x, y, z){
  plot(x, type = "l", col = 4, ylab = "Parameter Estimate", xlab = "Iteration",
       main = paste0("Traceplot for ", z))
  abline(h = y, col = 3, lwd = 2)
  },
  list(mcm1[[1]][, 1], mcm1[[1]][, 2], mcm1[[1]][, 3]), phi.mle,
  c("Intercept", "Phi 1", "Phi 2")))
```



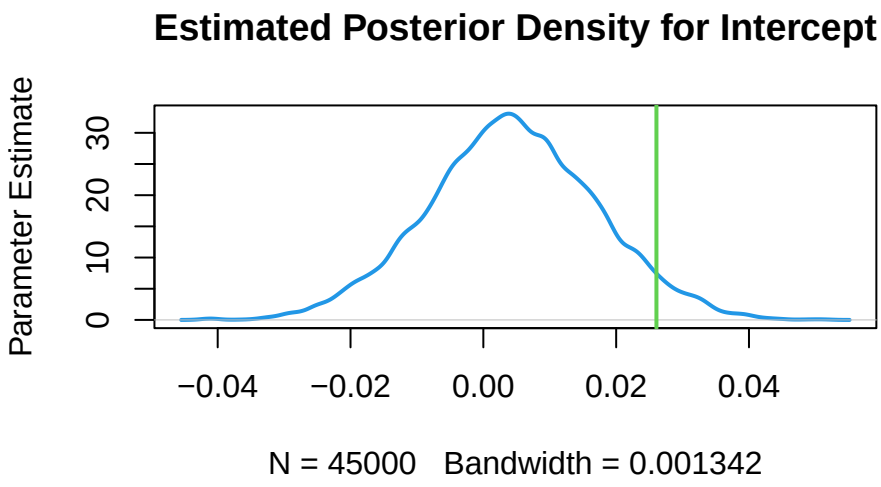


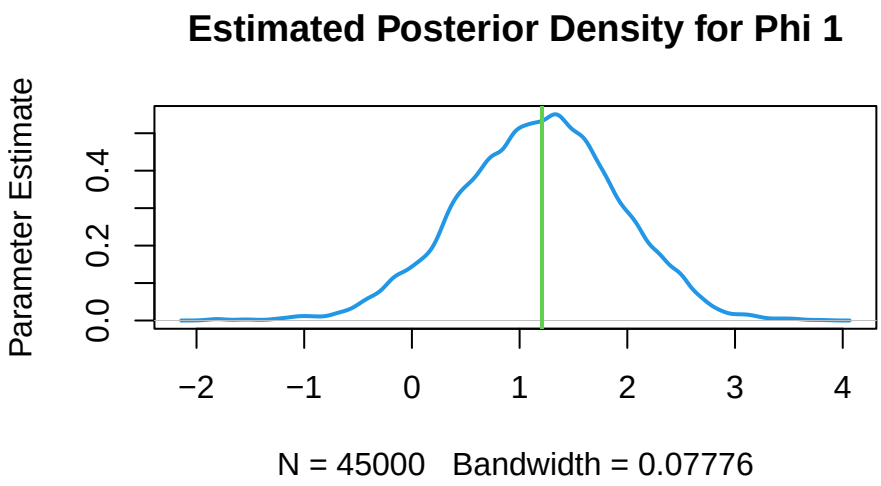


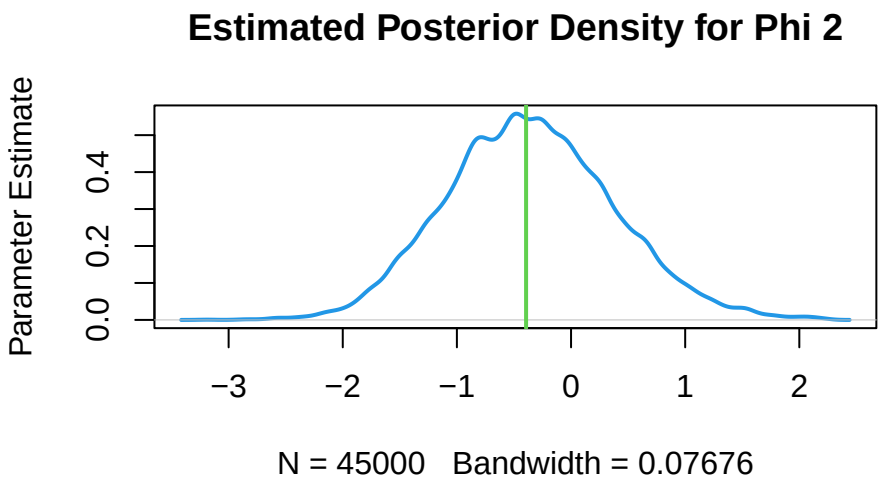

```
# and look at accept ratio
mcm1[[2]]
```

```
## [1] 0.1944364
```

```
# density
invisible(
Map(function(x, y, z){
  plot(density(x), type = "l", col = 4, lwd = 2,
       ylab = "Parameter Estimate", main = paste0("Estimated Posterior Density for ", z))
  abline(v = y, col = 3, lwd = 2)
  },
  list(mcm1[[1]][, 1], mcm1[[1]][, 2], mcm1[[1]][, 3]), phi.mle,
  c("Intercept", "Phi 1", "Phi 2")))
```


Estimated Posterior Density for Intercept
Parameter Estimate
0
10
20
30
−0.04
−0.02
0.00
0.02
0.04
N = 45000 Bandwidth = 0.001342


Estimated Posterior Density for Phi 1
Parameter Estimate
0.0
0.2
0.4
−2
−1
0
1
2
3
4
N = 45000 Bandwidth = 0.07776



The traceplots we obtain, nicely resemble caterpillars. This is a sign that the sampler converges very well. This is also confirmed by the accept ratio. Also, the posterior densities look very well behaved, i.e. they are unimodal and pretty symmetric. The estimates for φ_1, φ_2 are exactly centered at the Maximum Likelihood estimate (MLE). Thus, a frequentist approach would yield similar results. For the intercept, however, the MLE is only on the right tail of our posterior distribution.

7

To simulate the IRF for our AR(2) function, we can make use of the fact that any AR(p) process can be written as a vector AR(1) process. Once brought into the so called companion form we can easily obtain the IRF from the j'th power of the (1,1) element of the companion matrix.

```
# first exclude all non stable draws, i.e, |phi| > 1
bool <- apply(mcm1[[1]], 1, function(x) sum(abs(x)) < 1)




# subset
phi.clean <- mcm1[[1]][bool, ]

# median  and quantiles of estimates
est.med <- apply(phi.clean, 2, function(x){
  c(median(x), quantile(x, probs = c(0.6, 0.4)))
})



# Companion Matrix
phi.comp <- lapply(as.data.frame(t(est.med[, 2:3])), function(x){
  as.matrix(rbind(x, c(1, 0)))
})

# quick look at our
est.med
```

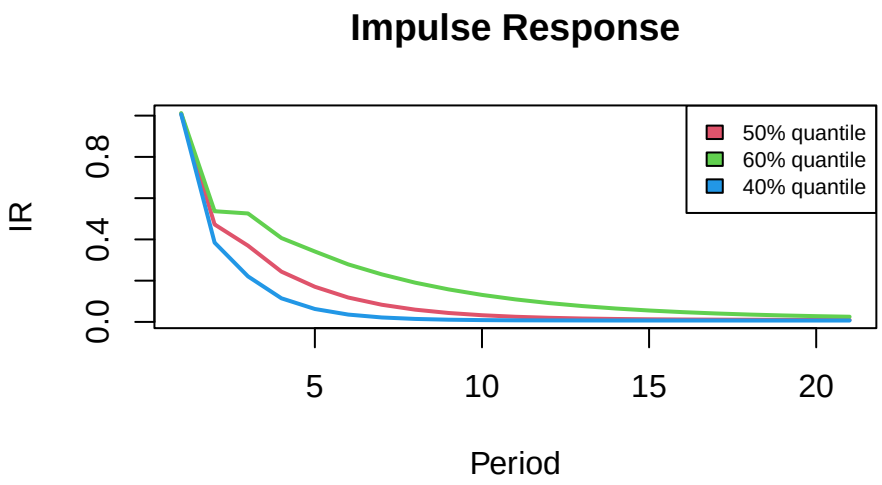
```
##            [,1]      [,2]      [,3]
```

```
##     0.009456129 0.4630176 0.1457653
## 60% 0.012342620 0.5242612 0.2385316
## 40% 0.006806870 0.3776736 0.0711981
```

```r
# number of periods and storage for impulse response
j <- 20
Imp.store <- matrix(NA, nrow = j + 1, ncol = 3)

# Now we can pick out the (1, 1) element of phi.comp^j to recover the impulse response
# at horizon j to a unit shock at time t. We can easily do this within a loop.
# We can use library(expm) to obtain a matrix power function.
library(expm)
```

```
## Lade nötiges Paket: Matrix

##
## Attache Paket: 'expm'

## Das folgende Objekt ist maskiert 'package:Matrix':
##
##     expm
```

```r
# cols for the plot
col <-  2:4

invisible(
Map(function(x, y, z){
  # storage
  st <- rep(NA, j + 1)

  # loop
  for(l in 0:20){
     st[l + 1] <- (x %^% l)[1, 1] + y
  }

  # plot
  if(x[1,1] == phi.comp[[1]][1, 1]){
    plot(st, type = "l", col = z, main = "Impulse Response", lwd = 2, ylab = "IR",
         xlab = "Period")
    legend("topright", fill = col, legend = c("50% quantile", "60% quantile", "40% quantile"),
           cex = 0.7)
  } else {
    lines(st, col = z, lwd = 2)
  }

}, phi.comp, est.med[, 1], col))
```



The unit shock applied to the AR(2) process transmits through our model as expected. The return to the initial value takes around twelve periods when using the median of our estimators distributions. The lower bound, for which we used the 40% quantile, is back to pre-shock levels after 9 periods and the upper bound is back to pre-shock levels after around 20 periods.

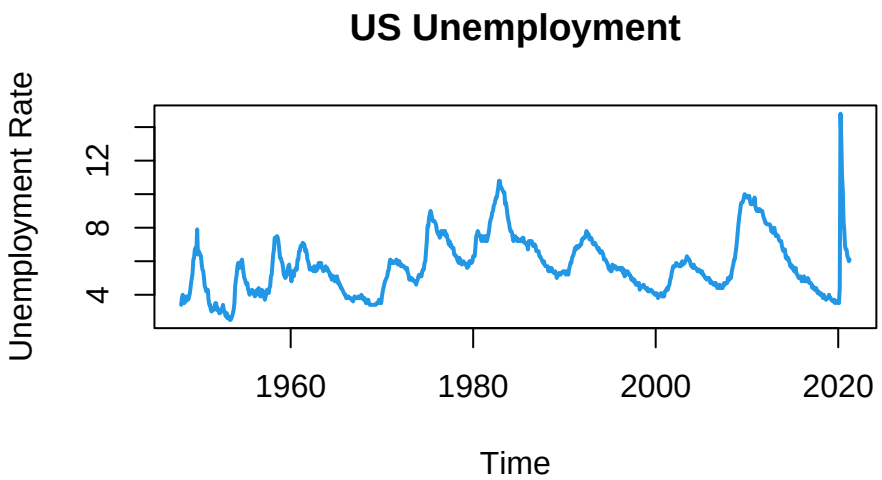
Prior Sensitivity

Again, we can use the **quantmod** package to download the data from the FRED database.

```
# Download data
quantmod::getSymbols("UNRATE", src = "FRED")
```

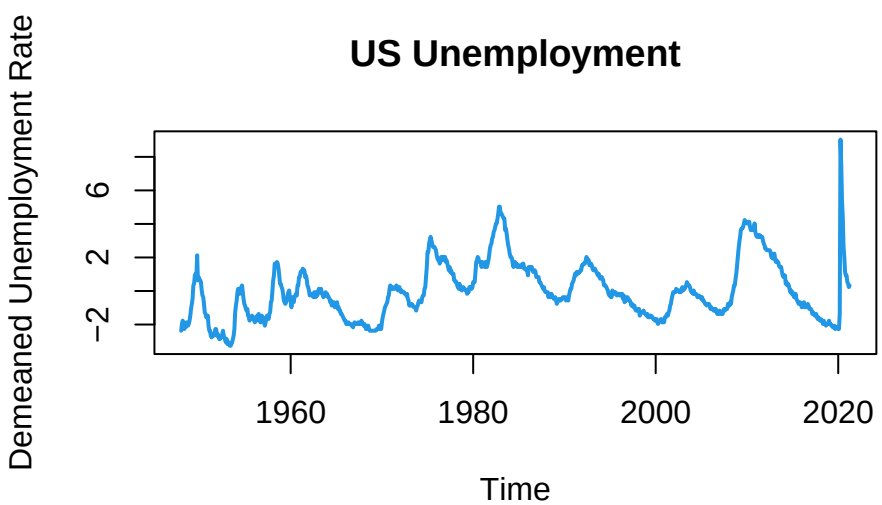
```
## [1] "UNRATE"
```

```
un.ts <- ts(UNRATE, start = c(1948, 1), frequency = 12)

# Plot
plot(un.ts, col = 4, lwd = 2, ylab = "Unemployment Rate", main = "US Unemployment")
```



1

```
# Demean series
un.demeaned <- un.ts - mean(un.ts)

# Plot
plot(un.demeaned, col = 4, lwd = 2, ylab = "Demeaned Unemployment Rate",
     main = "US Unemployment")
```



The demeaned time series is simply shifted downwards, by exactly the mean.

2

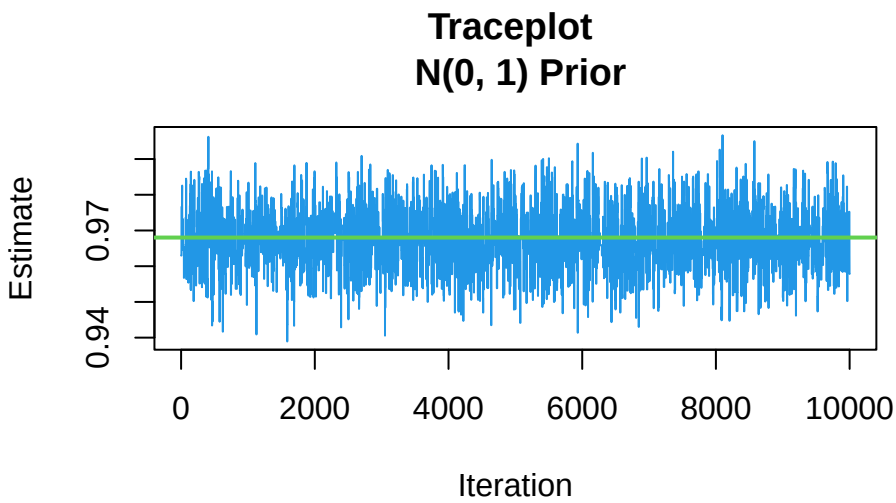
We will use the Metropolis algorithm as seen in the lecture.

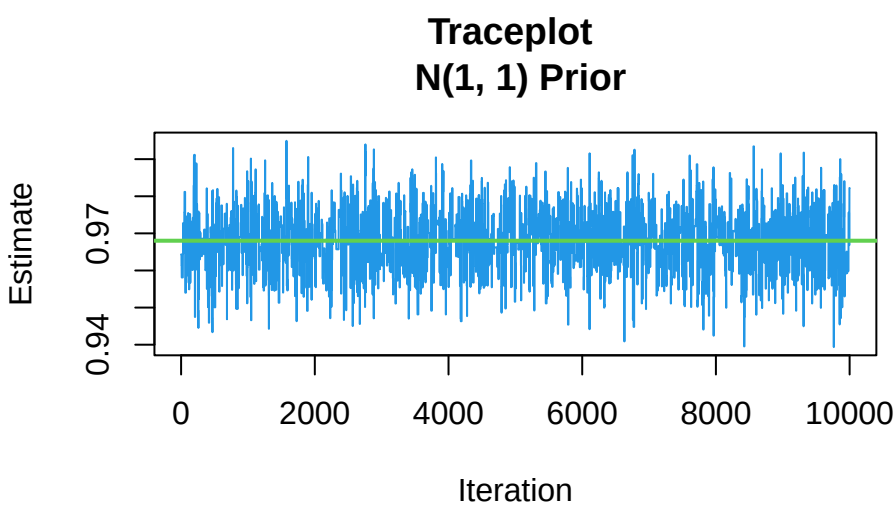
```
metro.est <- function(y, x, prior, prior.params){
```

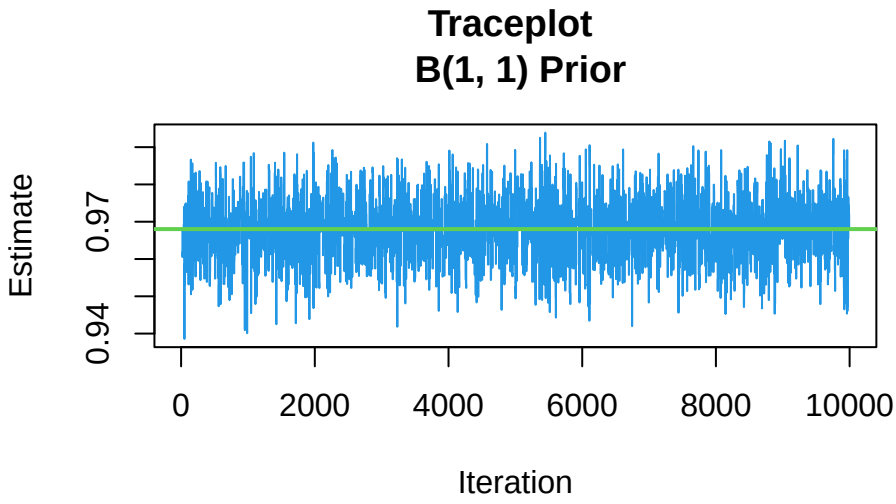
```r
# IG prior on sigma2
c0 <- 2
d0 <- 1

# Algorithm Setup
nburn <- 1e4
nsave <- 1e4
ntot <- nburn + nsave
t <- length(x)
eta2  <- 0.0161

sig2.draw <- 1
phi.draw  <- 0.5
accept    <- 0


# Storage matrix
phi.store <- matrix(NA, nsave, 1)

sig.store <- matrix(NA, nsave, 1)


# Metropolis algorithm loop
for (i in 1:ntot) {

  ## STEP 1:

  # Create proposal
  phi.prop <- rnorm(1, phi.draw, sqrt(eta2))

  # Evaluate prior for proposal
  prior.current <- do.call(paste0("d", prior),
                           list(phi.draw, prior.params[1], prior.params[2], log = TRUE))
  prior.prop    <- do.call(paste0("d", prior),
                           list(phi.prop, prior.params[1], prior.params[2], log = TRUE))

  # Evaluate LL
  ll.current <- sum(dnorm(y, x * phi.draw, sqrt(sig2.draw), log = TRUE))
  ll.prop <- sum(dnorm(y, x * phi.prop, sqrt(sig2.draw), log = TRUE))

  # And add prior
  c.p.current <- ll.current + prior.current
  c.p.prop <- ll.prop + prior.prop

  # And LR
  ll.ratio <- c.p.prop - c.p.current

  if(ll.ratio > log(runif(1))){ #if our proposal is likely "enough"

      #replace the current draw for rho with our proposal
      phi.draw <- phi.prop

      #add 1 to our acceptance counter
```

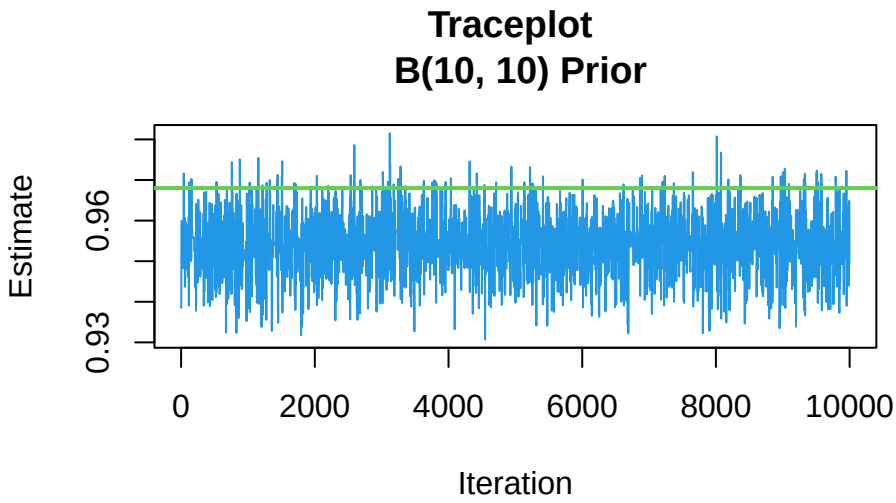
```r
        accept   <- accept + 1

      }

    # Tune proposal variance during first 50% of burn-in
    if (i < 0.5 * nburn) {
      if(accept / i > 0.3) eta2 <- eta2 * 1.01
      if(accept / i < 0.15) eta2 <- eta2 * 0.99
    }

    ## STEP 2: SAMPLE SIGMA2 GIVEN PHI & DATA

    # compute errors
    e  <- y - x * phi.draw

    # sum of squared residuals (equivalent to sum((e)^2))
    ee <- t(e) %*% e

    # compute posterior quantities
    ck <- c0 +  t/2
    dk <- d0 + ee/2
    # sample sigma2 from inverse gamma posterior
    sig2.draw <- 1 / rgamma(1, ck, dk)

    # STEP 3: STORE POSTERIOR SAMPLES

    #only after burn-in period!

    phi.store[i - nburn,]  <- phi.draw
    sig.store[i - nburn,] <- sig2.draw



  }

  return(list(Phi = phi.store, Sig2 = sig.store))

}

# Estimate posteriors
yy <- un.demeaned[-1]
xx <- un.demeaned[-length(un.demeaned)]

priors <- c("norm", "norm", "beta", "beta", "beta")
prior.params <- list(c(0,1), c(1, 1), c(1, 1), c(10, 10), c(0.1, 0.1))

res <- list()

for (i in 1:length(priors)) {
  res[[i]] <- metro.est(yy, xx, priors[i], prior.params[[i]])
}

# Estimate parameters in frequentist way for comparison
```

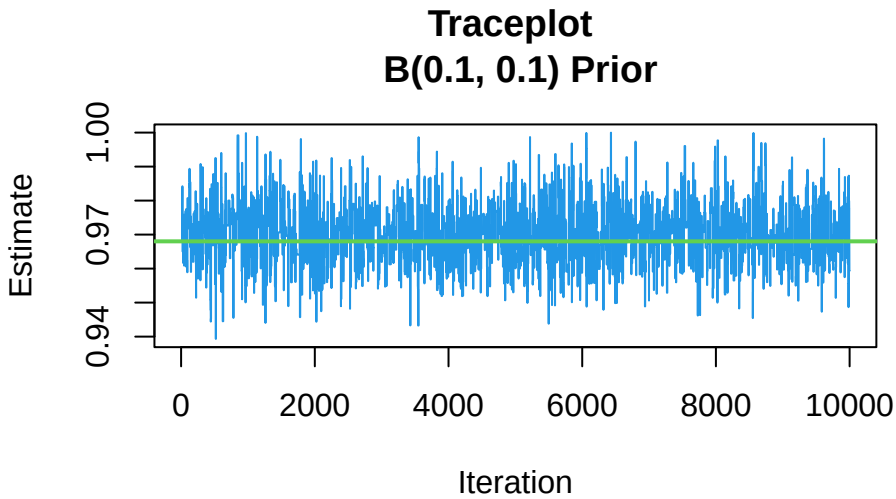
```r
phi.freq <- ar(yy, order.max = 1, method = "yule-walker")$ar

# traceplots
invisible(Map(function(x, y){
  plot.ts(x[[1]], xlab = "Iteration", ylab = "Estimate",
          main = paste("Traceplot", y), col = 4)
  abline(h = phi.freq, lwd = 2, col = 3)
  }, res,
  paste("\n", c("N(0, 1)", "N(1, 1)", "B(1, 1)", "B(10, 10)", "B(0.1, 0.1)"), "Prior")))
```






Traceplot
B(1, 1) Prior
Estimate
0.94
0.97
0
2000
4000
6000
8000
10000
Iteration

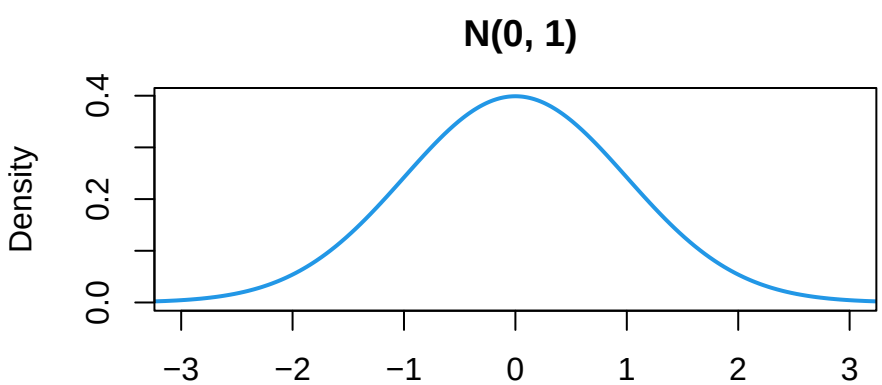

Traceplot
B(10, 10) Prior
Estimate
0.93
0.96
0
2000
4000
6000
8000
10000
Iteration

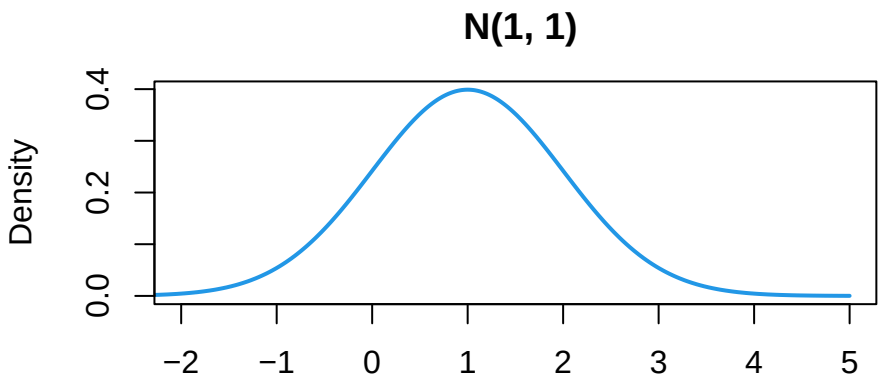


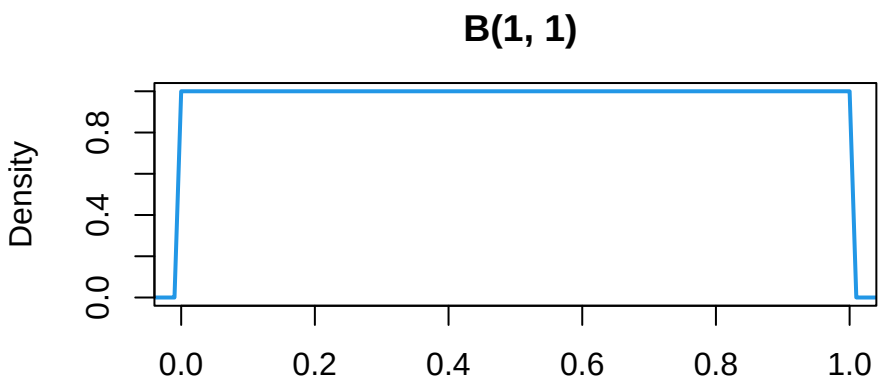
All the traceplots look like nice caterpillars. Thus, the sampler seems to converge well for all prior distributions.

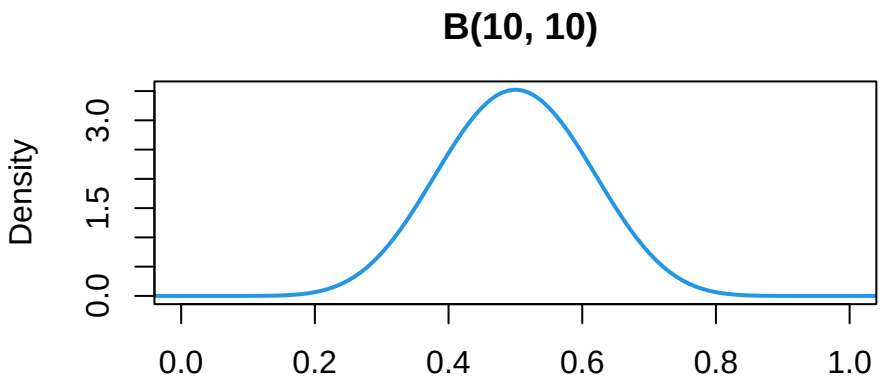
3

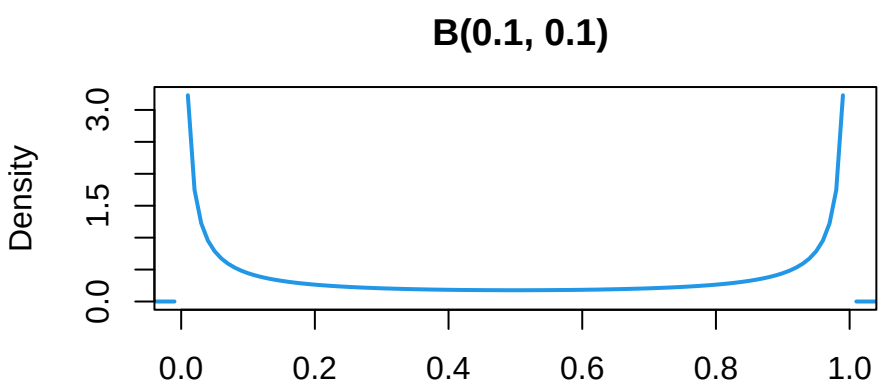
```
# Plot prior densities
invisible(Map(function(x, y, z, l){
  # support
  t <- seq(-5, 5, 0.01)

  # plot
  plot(t, do.call(paste0("d", x), list(t, y[1], y[2])), xlim = z,
       type = "l", lwd = 2, col = 4, ylab = "Density", xlab = "", main = l)
}, priors, prior.params, list(c(-3, 3), c(-2, 5), c(0, 1), c(0, 1), c(0, 1)),
paste("\n", c("N(0, 1)", "N(1, 1)", "B(1, 1)", "B(10, 10)", "B(0.1, 0.1)"))))
```




N(1, 1)
Density
0.0
0.2
0.4
−2
−1
0
1
2
3
4
5


B(1, 1)
Density
0.0
0.4
0.8
0.0
0.2
0.4
0.6
0.8
1.0





A standard normal prior (with low values for σ) for the autoregressive coefficient is rather informative and implies that the resulting AR(1) process will be stationary since the interval (-1, 1) has a high probability mass in the support of a standard normal distribution.

A normal prior that is centered around 1 with $\sigma^2 = 1$ is also rather informative and will most likely yield an AR(1) that is stationary.

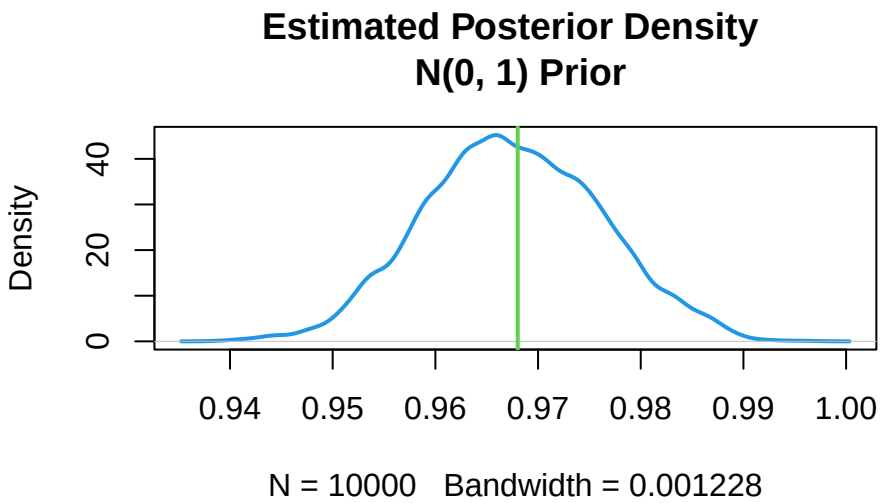
A beta distribution with the parameters $\alpha = \beta = 1$ corresponds to a uniform distribution on the interval [0, 1]. Thus, such a prior is suitable for the autoregressive coefficient of a stationary AR(1).

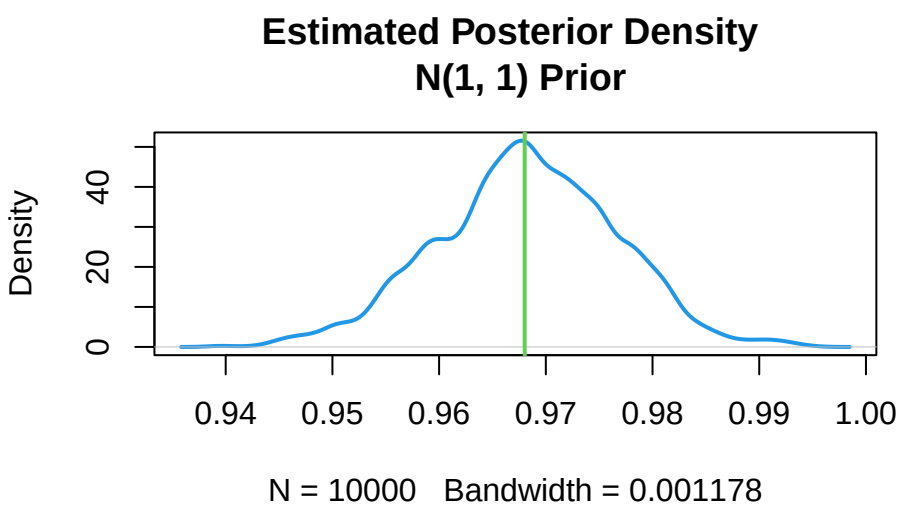
The PDF of a beta distribution is approximately normal if the sum of its parameters α and β are large enough and $\alpha \approx \beta$, which is the case for the given parameters $\alpha = \beta = 10$. This prior is more informative than the other priors since it has a smaller support. As can be seen from the plot, the bell-shaped density is centred around 0.5 and ranges from 0.2 to 0.8. Thus, the prior will yield a stationary AR(1).

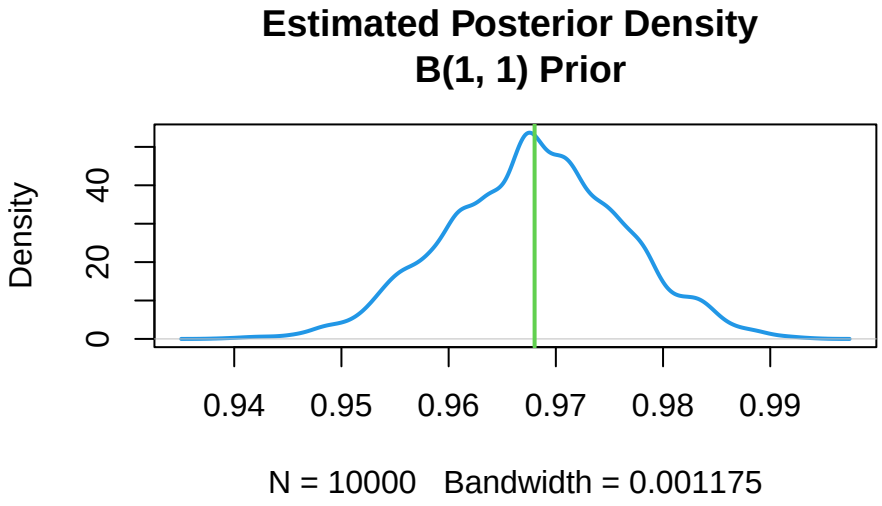
With a smaller domain, this prior is also more informative than the first three. However, like the other priors, one can expect to obtain a stationary AR(1) given the support of the prior.

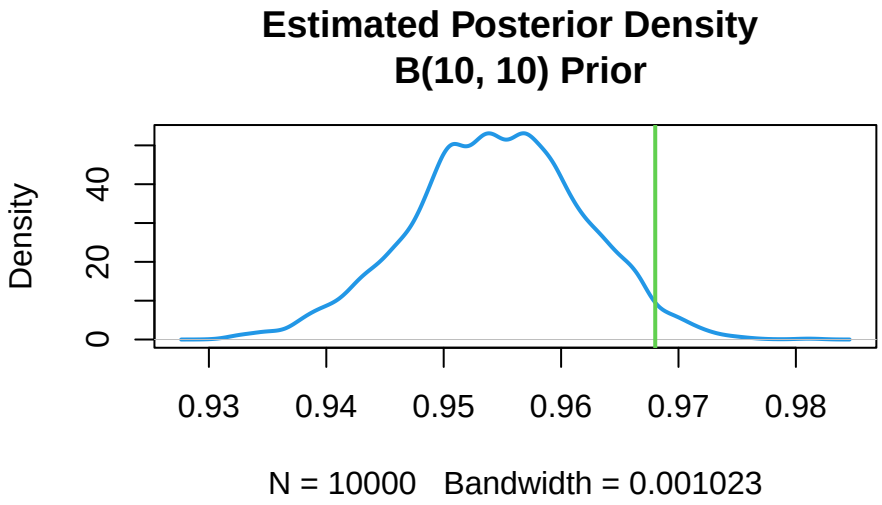
4

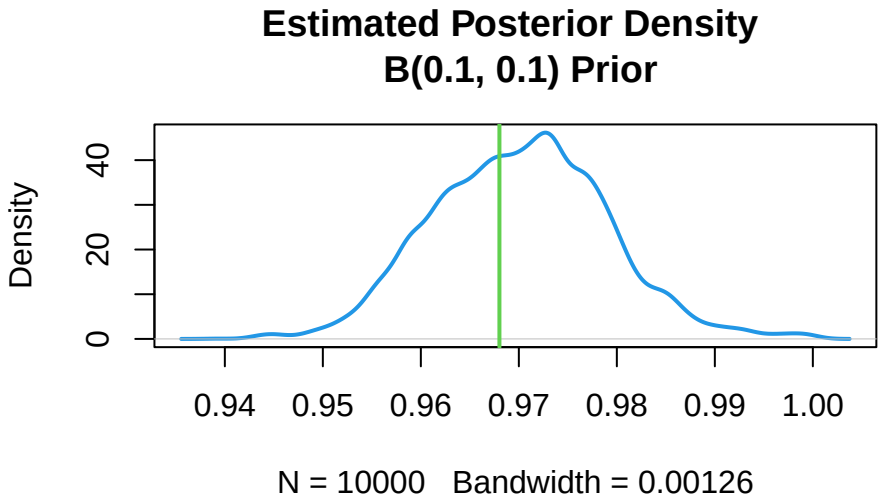
```r
# posterior densities
invisible(Map(function(x, y){
  plot(density(x[[1]]), lwd = 2, col = 4, main = paste("Estimated Posterior Density", y))
  abline(v = phi.freq, lwd = 2, col = 3)
  }, res,
  paste("\n", c("N(0, 1)", "N(1, 1)", "B(1, 1)", "B(10, 10)", "B(0.1, 0.1)"), "Prior")))
```






Estimated Posterior Density
B(1, 1) Prior
Density
0
20
40
0.94
0.95
0.96
0.97
0.98
0.99
N = 10000 Bandwidth = 0.001175


Estimated Posterior Density
B(10, 10) Prior
Density
0
20
40
0.93
0.94
0.95
0.96
0.97
0.98
N = 10000 Bandwidth = 0.001023



The vertical green line is the value obtained for the autoregressive coefficient estimating the model by the Yule-Walker equations. The posterior results for the priors N(0, 1), N(1, 1) and B(1, 1) do not differ very much, thus it can be concluded that they are not sensitive to the prior assumptions. This is not very surprising since the priors a) and b) are from the same distribution with slightly different parameters. The c) prior is equivalent to a uniform distribution for the interval [0, 1], which is in general a suitable behaviour for an autoregressive coefficient which is stationary. The Beta prior with parameters $\alpha = 10$ and $\beta = 10$ on the other hand yields a posterior that is centered around a lower value than the priors before. With a starting value of zero for the autoregressive coefficient, the mean of this posterior is rather far from the estimated value (0.967) obtained from the frequentist estimation. This is due to the fact that the prior is very informative given its small support. A strong prior tends to push away the estimated coefficient from its true value by overwriting the information in the likelihood given by the data. Thus, the posterior mean is pushed away from 0.967 towards zero.

Regarding the prior B(.1, .1), the resulting posterior depends on the starting value for the autoregressive coefficient. A starting value of 1 yields a perfectly bell-shaped posterior density and a horizontal line as a trace plot. Whereas a starting value of 0.5 results in a well-behaved posterior with a "jigsaw"-shaped trace plot. An explanation for this result can be given by looking at the plot of the prior: 0 and 1 have a very high density, and therefore one tends to stay around the same area if the starting value is set to either 0 or 1. With a starting value other than zero or one for the autoregressive coefficient, we obtain a similar posterior as in a)-c).